CANVIA

CAPACITACIÓN QA AUTOMATION · PARTE 1 DE 2

MANUAL DE INSTALACIÓN

Automatización de APIs con REST Assured + Cucumber

Del equipo en blanco al primer escenario BDD en verde: Java, Maven, el IDE y un proyecto base que ya fue ejecutado contra una API real antes de llegar a tus manos.

Java 17+
JDK Temurin LTS

Maven 3.9
dependencias y ejecución

REST Assured 6
cliente HTTP para pruebas

Cucumber 7
escenarios en Gherkin

01 Antes de empezar: qué vas a instalar
02 Paso 1 · Java (JDK)
03 Paso 2 · Maven
04 Paso 3 · VS Code y sus extensiones
05 Paso 4 · Crear el proyecto
06 Paso 5 · El archivo pom.xml
07 Paso 6 · Runner, hooks y contexto
08 Paso 7 · El primer feature y sus steps
09 Paso 8 · Ejecutar y leer el reporte
10 Errores típicos y su solución
11 Checklist final

Instructor: **Giancarlo Palomino** · QA Automation Engineer

Windows · macOS · Linux · v1.0 · octubre 2026

MAPA DE LA INSTALACIÓN

Qué vas a instalar y para qué sirve cada pieza

Piensa en una cocina: Java es la electricidad, Maven es el proveedor que trae los ingredientes, el IDE es la mesa de trabajo, REST Assured es el cuchillo y Cucumber es la receta que cualquiera puede leer. Sin una de ellas no sale el plato.

1 · BASE → **JDK 17** — ejecuta código Java

2 · CONSTRUCCIÓN → **Maven** — descarga librerías y corre pruebas

3 · TRABAJO → **VS Code** — editas, ejecutas, depuras

4 · PROYECTO → **pom.xml** — REST Assured + Cucumber + JUnit

PIEZA	PARA QUÉ SIRVE	VERSIÓN PROBADA	CÓMO LLEGA
JDK Temurin	El motor que compila y ejecuta Java. Usamos una versión LTS (soporte largo).	`17` o `21`	Instalador
Apache Maven	Lee el `pom.xml`, descarga las librerías de Maven Central y ejecuta las pruebas con `mvn test`.	`3.9.x`	Instalador / zip
VS Code	El editor del curso (Java y TypeScript): autocompletado, terminal integrada y salto del step al código con sus extensiones.	1.105 o superior	Instalador
REST Assured	Envía peticiones HTTP y valida las respuestas con una sintaxis `given / when / then`.	`6.3.0`	pom.xml
Cucumber JVM	Lee los archivos `.feature` escritos en Gherkin y los conecta con métodos Java.	`7.34.9`	pom.xml
JUnit Platform	El "motor de arranque" que Maven usa para descubrir y correr los escenarios.	`6.1.3`	pom.xml
Jackson	Convierte objetos Java a JSON (y al revés) para los cuerpos de las peticiones.	`2.20.0`	pom.xml
PicoContainer	Comparte datos entre clases de steps dentro del mismo escenario.	vía BOM	pom.xml

REQUISITOS DEL EQUIPO

Windows 10/11, macOS 13+ o Linux (Ubuntu 22.04+, Debian 12+, Fedora 39+) · 8 GB de RAM · 3 GB libres · conexión a internet sin bloqueo a `repo1.maven.org` (si estás detrás del proxy corporativo, revisa la página de errores).

PROYECTO VERIFICADO

Todo el código de este manual se ejecutó el 26-10-2026 contra `restful-booker.herokuapp.com`: 4 escenarios, 0 fallos, `BUILD SUCCESS` en 7 segundos.

BASE

1 Instalar Java (JDK 17 o 21)

REST Assured 6 y JUnit 6 exigen **Java 17 como mínimo**. Usa la columna de tu sistema operativo. Si ya tienes un Java 8 antiguo (muy común en laptops corporativas), no lo desinstales: basta con que el nuevo quede primero.

Windows 10 / 11

1 · INSTALAR

```
# PowerShell
winget install
EclipseAdoptium.Temurin.17.JDK
```

O el instalador `.msi` de `adoptium.net` marcando **"Set JAVA_HOME variable"**.

2 · VARIABLES (SI NO QUEDARON)

```
$jdk = "C:\Program
Files\Eclipse Adoptium\jdk-
17.0.17.10-hotspot"
setx JAVA_HOME "$jdk"
```

Luego, en *Variables de entorno → Path*, agrega `%JAVA_HOME%\bin` **arriba** de cualquier ruta de Oracle Java.

3 · VERIFICAR (TERMINAL NUEVA)

```
java -version
echo $env:JAVA_HOME
```

macOS INTEL / APPLE SILICON

1 · INSTALAR (HOMEBREW)

```
brew install --cask temurin@17
```

¿No tienes Homebrew? Instálalo desde `brew.sh`. Alternativa: el `.pkg` de `adoptium.net`.

2 · VARIABLES

```
# agrega al final de ~/.zshrc
export
JAVA_HOME=$(/usr/libexec/java_
home -v 17)
export
PATH="$JAVA_HOME/bin:$PATH"
```

Aplica sin cerrar la terminal con `source ~/.zshrc`.

3 · VERIFICAR

```
java -version
echo $JAVA_HOME
```

Linux UBUNTU · DEBIAN · FEDORA

1 · INSTALAR

```
# Ubuntu / Debian
sudo apt update
sudo apt install openjdk-17-
jdk
# Fedora / RHEL
sudo dnf install java-17-
openjdk-devel
```

2 · VARIABLES

```
# agrega al final de ~/.bashrc
export JAVA_HOME=$(dirname
$(dirname $(readlink -f
$(which javac))))
export
PATH="$JAVA_HOME/bin:$PATH"
```

Si conviven varios Java: `sudo update-alternatives --config java`.

3 · VERIFICAR

```
java -version
echo $JAVA_HOME
```

```
# Resultado esperado (igual en los tres sistemas)
$ java -version
openjdk version "17.0.17" 2025-10-21
OpenJDK Runtime Environment Temurin-17.0.17+10
```

SI VES "1.8.0"

Hay un Java 8 delante en el `PATH`. Windows: sube `%JAVA_HOME%\bin` en el Path. macOS/Linux: revisa que el `export` esté en el archivo correcto y abre otra terminal.

OPCIÓN MULTIPLATAFORMA: SDKMAN

En macOS, Linux o WSL, `curl -s "https://get.sdkman.io" | bash` y luego `sdk install java 17.0.17-tem` y `sdk install maven`: instala ambos y configura las variables por ti.

CONSTRUCCIÓN

2 Instalar Apache Maven 3.9

Maven lee la "lista de compras" del proyecto (`pom.xml`), descarga cada librería a la carpeta `~/.m2` la primera vez y ejecuta las pruebas.

Windows — ZIP O GESTOR

OPCIÓN A · MANUAL

1. Descarga **apache-maven-3.9.x-bin.zip** de `maven.apache.org` .
2. Descomprime en `C:\tools\` (ruta sin espacios).
3. Crea `MAVEN_HOME` con esa ruta.
4. Agrega `%MAVEN_HOME%\bin` al Path.

OPCIÓN B · GESTOR

```
scoop install maven
# o bien
choco install maven
```

VERIFICAR (TERMINAL NUEVA)

```
mvn -v
```

macOS — HOMEBREW

INSTALAR

```
brew install maven
```

DETALLE DE HOMEBREW

La fórmula de Maven trae su propio OpenJDK. Maven usará Temurin 17 solo si `JAVA_HOME` está definido en `~/.zshrc` (Paso 1).

VERIFICAR

```
mvn -v
```

La línea *Java version* debe decir 17 (o 21) y la ruta debe ser la de Temurin.

Linux — APT · DNF · SDKMAN

INSTALAR

```
# Ubuntu / Debian
sudo apt install maven
# Fedora / RHEL
sudo dnf install maven
# versión más reciente
sdk install maven
```

Los repositorios de algunas distribuciones traen Maven 3.8: también sirve para este proyecto; SDKMAN da la 3.9.

VERIFICAR

```
mvn -v
```

```
$ mvn -v
Apache Maven 3.9.9 (8e8579a9e76f7d015ee5ec7bfcdc97d260186937)
Java version: 17.0.17, vendor: Eclipse Adoptium, runtime: …/jdk-17.0.17+10
```

MIRA LA LÍNEA "JAVA VERSION"

Maven usa el Java de `JAVA_HOME` , no el que responde `java -version` . Si aquí aparece `1.8` , la compilación fallará con `invalid target release: 17` .

DETRÁS DEL PROXY CORPORATIVO (LOS TRES SISTEMAS)

Si Maven no descarga nada (*Connect timed out*), crea `~/.m2/settings.xml` (en Windows: `%USERPROFILE%\.m2\settings.xml`) con un bloque `<proxies><proxy>` indicando `host` y `port` que te dé Mesa de Ayuda.

MESA DE TRABAJO

3 Visual Studio Code para Java y Cucumber

Usamos **el mismo editor en las dos partes del curso**: VS Code sirve para Java (esta parte) y para TypeScript (la parte de Playwright). Un solo editor, unos mismos atajos, y nada que reaprender al cambiar de tema.

1 · Instalar VS Code

Windows — WINGET

```
winget install Microsoft.VisualStudioCode
```

macOS — HOMEBREW

```
brew install --cask visual-studio-code
```

Linux — SNAP · O .DEB/.RPM

```
sudo snap install code --classic
```

2 · Instalar las extensiones

Desde la terminal (igual en los tres sistemas) o buscándolas en la vista de extensiones (`Ctrl`+`Shift`+`X` · `⌘`+`⇧`+`X`):

Terminal — INSTALAR

```
code --install-extension vscjava.vscode-java-pack
code --install-extension CucumberOpen.cucumber-official
```

- **Extension Pack for Java** (Microsoft): lenguaje, depurador, *Maven for Java* y *Test Runner for Java*.
- **Cucumber** (CucumberOpen): colores y autocompletado en los `.feature`, y salto del paso a su método Java.

3 · Decirle a Cucumber dónde está todo

.vscode/settings.json — EN EL PROYECTO

```
{
  "cucumber.features": [
    "src/test/resources/features/**/*.feature"
  ],
  "cucumber.glue": [
    "src/test/java/**/*.java"
  ]
}
```

4 · Abrir el proyecto

File → Open Folder sobre la carpeta que contiene el `pom.xml`. Abajo a la derecha aparecerá *"Java: Ready"* cuando termine de importar Maven (la primera vez, 1–3 min).

EL JDK DEL EDITOR

La extensión de Java usa el JDK de `JAVA_HOME` para compilar el proyecto. Si muestra *"Java 17 or more recent is required"* o marca errores raros, revisa el Paso 1 y ejecuta *Java: Clean Java Language Server Workspace* desde la paleta de comandos.

ATAJOS QUE USARÁS A DIARIO (WINDOWS/LINUX · MACOS)

Ir del paso a su método: `F12` o `Ctrl`+clic · `⌘`+clic
Paleta de comandos: `Ctrl`+`Shift`+`P` · `⌘`+`⇧`+`P`
Terminal integrada: `Ctrl`+`ñ` o `Ctrl`+`` ` `` · `^`+`` ` ``

CÓMO EJECUTAR

La forma más fiable, y la misma que usará el pipeline de CI, es la terminal integrada: `mvn test`. También puedes abrir la vista **Testing** (ícono del matraz) y ejecutar `RunCucumberTest` con ▶.

¿PREFIERES INTELLIJ IDEA?

También es válido: `winget install JetBrains.IntelliJIDEA.Community`, `brew install --cask intellij-idea-ce` o `sudo snap install intellij-idea-community --classic`, más el plugin **Cucumber for Java**. Permite ejecutar un escenario suelto con ▶ desde el feature.

ESQUELETO

4 Crear el proyecto y su estructura

Crea una carpeta vacía, ábrela en VS Code y crea las carpetas a mano (o descomprime el proyecto base que acompaña este manual). Lo importante es respetar la convención de Maven: el código de prueba vive en `src/test/java` y los recursos (features) en `src/test/resources`.

```
api-bdd-restassured/
├── pom.xml                          # dependencias y plugins
└── src/test/
    ├── java/com/canvia/qa/
    │   ├── runners/
    │   │   └── RunCucumberTest.java    # arranca Cucumber
    │   ├── steps/
    │   │   └── ReservaSteps.java       # Gherkin → Java
    │   └── support/
    │       ├── Hooks.java              # antes/después
    │       └── Contexto.java           # datos del escenario
    └── resources/
        ├── junit-platform.properties
        └── features/
            └── reservas.feature        # escenarios
```

REGLA DE ORO

El paquete del runner y de los steps debe estar **dentro** del paquete que declaras como `glue` (aquí `com.canvia.qa`). Si los steps quedan fuera, Cucumber dirá que no existen.

¿POR QUÉ LA CARPETA FEATURES VA EN RESOURCES?

Porque los `.feature` no se compilan: Maven los copia tal cual al classpath y Cucumber los busca allí como si fueran un paquete llamado `features`.

NOMBRE DEL RUNNER

Surefire solo ejecuta clases cuyo nombre termina en `Test` (o empieza con `Test`). Si lo llamas `CucumberRunner` verás *"No tests to run"*.

La API que vamos a probar

Restful Booker (`restful-booker.herokuapp.com`) es una API pública de reservas de hotel creada para practicar pruebas. Tiene CRUD completo, autenticación por token y — a propósito — algunos bugs que cazaremos en los ejercicios.

GET	`/ping` · `/booking/{id}`
POST	`/booking` · `/auth`
PUT / PATCH	`/booking/{id}` (requiere token)
DELETE	`/booking/{id}` (requiere token)

Tres ideas para leer el pom.xml de la página siguiente

PROPERTIES

`release 17` compila para Java 17. `UTF-8` evita que "código" se lea como "cÃ³digo". Las versiones quedan en un solo lugar.

BOM

Un *bill of materials* fija versiones compatibles entre sí. Por eso Cucumber y JUnit no llevan `<version>` propia.

JACKSON NO ES OPCIONAL

REST Assured no trae serializador JSON. Sin `jackson-databind`, enviar un `Map` como body falla.

LISTA DE COMPRAS

5 El archivo pom.xml completo

pom.xml — COPIAR TAL CUAL

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.canvia.qa</groupId>
  <artifactId>api-bdd-restassured</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <cucumber.version>7.34.9</cucumber.version>
    <junit.version>6.1.3</junit.version>
    <restassured.version>6.3.0</restassured.version>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>io.cucumber</groupId><artifactId>cucumber-bom</artifactId><version>${cucumber.version}</version>
        <type>pom</type><scope>import</scope>
      </dependency>
      <dependency>
        <groupId>org.junit</groupId><artifactId>junit-bom</artifactId><version>${junit.version}</version>
        <type>pom</type><scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <dependencies>
    <dependency>
      <groupId>io.rest-assured</groupId><artifactId>rest-assured</artifactId>
      <version>${restassured.version}</version><scope>test</scope>
    </dependency>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId><artifactId>jackson-databind</artifactId>
      <version>2.20.0</version><scope>test</scope>
    </dependency>
    <dependency>
      <groupId>io.cucumber</groupId><artifactId>cucumber-java</artifactId><scope>test</scope>
    </dependency>
    <dependency>
      <groupId>io.cucumber</groupId><artifactId>cucumber-junit-platform-engine</artifactId><scope>test</scope>
    </dependency>
    <dependency>
      <groupId>io.cucumber</groupId><artifactId>cucumber-picocontainer</artifactId><scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.junit.platform</groupId><artifactId>junit-platform-suite</artifactId><scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId><artifactId>maven-surefire-plugin</artifactId>
        <version>3.5.4</version>
      </plugin>
    </plugins>
  </build>
</project>
```

DESPUÉS DE PEGAR

Guarda el archivo: VS Code detecta el cambio y Maven descarga las librerías (si pregunta, elige **Always** para sincronizar la configuración). La primera descarga tarda de 1 a 3 minutos. `scope test` indica que nada de esto viaja a producción.

LAS PIEZAS QUE NO SE VEN

6 Runner, propiedades, hooks y contexto

runners/RunCucumberTest.java — ARRANQUE

```java
package com.canvia.qa.runners;

import org.junit.platform.suite.api.*;
import static io.cucumber.junit.platform.engine
              .Constants.*;

@Suite
@IncludeEngines("cucumber")
@SelectPackages("features")
@ConfigurationParameter(key =
GLUE_PROPERTY_NAME,
        value = "com.canvia.qa")
@ConfigurationParameter(key =
PLUGIN_PROPERTY_NAME,
        value = "pretty, html:target/cucumber-
report.html")
public class RunCucumberTest {
}
```

Cómo leerlo: "JUnit, usa el motor de Cucumber (`@IncludeEngines`), busca los features en el paquete `features` , el código de los pasos está en `com.canvia.qa` y genera un reporte HTML".

resources/junit-platform.properties — OPCIONAL

```
cucumber.publish.quiet=true
```

Silencia el aviso que invita a publicar reportes en la nube de Cucumber.

support/Hooks.java — ANTES DE CADA ESCENARIO

```java
package com.canvia.qa.support;

import io.cucumber.java.Before;
import io.restassured.RestAssured;

public class Hooks {

    @Before
    public void configurarApi() {
        String url = "https://restful-
booker.herokuapp.com";
        RestAssured.baseURI =
System.getProperty("baseUrl", url);
        // Si una validación falla, imprime
petición y respuesta
        RestAssured

.enableLoggingOfRequestAndResponseIfValidationF
ails();
    }
}
```

support/Contexto.java — LA MOCHILA DEL ESCENARIO

```java
package com.canvia.qa.support;

import io.restassured.response.Response;

/** Estado compartido entre los steps de un
mismo escenario. */
public class Contexto {
    public Response respuesta;
    public Integer idReserva;
}
```

Qué pasa cuando escribes `mvn test`

MAVEN — **Surefire** — busca clases *Test → JUNIT — **@Suite** — lee el runner → CUCUMBER — **Motor** — carga features y glue → POR ESCENARIO — **@Before** — URL base y Contexto nuevo → STEPS — **REST Assured** — HTTP real a la API

@BEFORE
Corre antes de **cada** escenario. Aquí fijamos la URL base, que se puede cambiar con `-DbaseUrl=…` sin tocar código.

LOG SI FALLA
Si una validación falla, REST Assured imprime la petición y la respuesta completas. Es tu primera pista al depurar.

CONTEXTO
PicoContainer crea uno nuevo por escenario y lo inyecta en el constructor de cada clase de steps: así un paso guarda y otro lee.

LO QUE SÍ SE VE

7 El primer feature, en español

Un archivo `.feature` describe el comportamiento con palabras de negocio. La primera línea, `# language: es`, le dice a Cucumber que las palabras clave vienen en español: *Característica, Escenario, Dado, Cuando, Entonces, Y*.

features/reservas.feature — GHERKIN

```gherkin
# language: es
Característica: Gestión de reservas en Restful Booker
  Como cliente de la API de reservas
  Quiero crear y consultar reservas
  Para gestionar las estadías de los huéspedes

  Antecedentes:
    Dado que la API de reservas está disponible

  @smoke
  Escenario: Consultar una reserva recién creada
    Dado que existe una reserva a nombre de "Rosa"
    Cuando consulto esa reserva
    Entonces el código de respuesta es 200
    Y la respuesta contiene el campo "firstname"

  @regresion
  Esquema del escenario: Crear reservas
    Cuando creo una reserva para "<nombre>" "<apellido>" por <precio
    Entonces el código de respuesta es 200
    Y el nombre de la reserva creada es "<nombre>"

    Ejemplos:
      | nombre | apellido | precio |
      | Ana    | Torres   | 150    |
      | Luis   | Quispe   | 320    |

  @seguridad
  Escenario: No se puede eliminar sin token
    Cuando elimino la reserva con id 1 sin autenticarme
    Entonces el código de respuesta es 403
```

CARACTERÍSTICA
El tema del archivo y, debajo, la historia de usuario: quién, qué y para qué.

ANTECEDENTES
Pasos que se repiten antes de cada escenario del archivo.

@ETIQUETAS
Sirven para filtrar: `@smoke` lo crítico, `@regresion` todo.

ESQUEMA + EJEMPLOS
Se ejecuta una vez por fila. Dos filas = dos pruebas.

Cómo encuentra Cucumber el código de cada paso

TEXTO DEL FEATURE
`creo una reserva para "Ana" "Torres" por 150 soles`

→

EXPRESIÓN DEL STEP
`creo una reserva para {string} {string} por {int} soles`

→

PARÁMETROS DEL MÉTODO
`("Ana", "Torres", 150)`

REGLA PARA NO SUFRIR
El texto debe coincidir letra por letra, tildes incluidas. `{string}` captura lo que va entre comillas; `{int}`, un número entero. El resto de la frase es literal.

GHERKIN → JAVA

7 Las step definitions

ENCABEZADO DEL ARCHIVO (PAQUETE E IMPORTS)

`package com.canvia.qa.steps;` · `io.cucumber.java.es.Dado / Cuando / Entonces` (anotaciones en español) · `static io.restassured.RestAssured.given` · `static org.hamcrest.Matchers.equalTo / notNullValue` · `io.restassured.http.ContentType` · `java.util.Map` · `com.canvia.qa.support.Contexto`

steps/ReservaSteps.java — JAVA

```java
public class ReservaSteps {
    private final Contexto contexto;

    // PicoContainer inyecta el mismo Contexto en todas las clases de steps
    public ReservaSteps(Contexto contexto) { this.contexto = contexto; }

    @Dado("que la API de reservas está disponible")
    public void apiDisponible() {
        given().when().get("/ping").then().statusCode(201);
    }

    @Dado("que existe una reserva a nombre de {string}")
    public void existeReserva(String nombre) {
        creoReserva(nombre, "Prueba", 100);
        contexto.idReserva = contexto.respuesta.then().statusCode(200).extract().path("bookingid");
    }

    @Cuando("consulto esa reserva")
    public void consultoReserva() {
        contexto.respuesta = given()
                .accept("application/json")
                .pathParam("id", contexto.idReserva)
            .when()
                .get("/booking/{id}");
    }

    @Cuando("creo una reserva para {string} {string} por {int} soles")
    public void creoReserva(String nombre, String apellido, int precio) {
        Map<String, Object> cuerpo = Map.of(
            "firstname", nombre, "lastname", apellido,
            "totalprice", precio, "depositpaid", true,
            "bookingdates", Map.of("checkin", "2026-11-01",
                                   "checkout", "2026-11-05"));
        contexto.respuesta = given()
                .contentType(ContentType.JSON)
                .accept("application/json")
                .body(cuerpo)
            .when()
                .post("/booking");
    }

    @Cuando("elimino la reserva con id {int} sin autenticarme")
    public void eliminoSinToken(int id) {
        contexto.respuesta = given().pathParam("id", id).when().delete("/booking/{id}");
    }

    @Entonces("el código de respuesta es {int}")
    public void codigoDeRespuesta(int codigo) {
        contexto.respuesta.then().statusCode(codigo);
    }

    @Entonces("la respuesta contiene el campo {string}")
    public void contieneCampo(String campo) {
        contexto.respuesta.then().body(campo, notNullValue());
    }

    @Entonces("el nombre de la reserva creada es {string}")
    public void nombreCreado(String nombre) {
        contexto.respuesta.then().body("bookingid", notNullValue())
            .body("booking.firstname", equalTo(nombre));
    }
}
```

LUZ VERDE

8 Ejecutar y leer el resultado

COMANDO (IGUAL EN WINDOWS, MACOS Y LINUX)	QUÉ HACE
`mvn test`	Compila y ejecuta todos los escenarios.
`mvn test "-Dcucumber.filter.tags=@smoke"`	Solo los escenarios con la etiqueta `@smoke`.
`mvn test "-Dcucumber.filter.tags=not @seguridad"`	Todos menos los de seguridad. También vale `and` / `or`.
`mvn test "-DbaseUrl=http://localhost:3001"`	Misma suite contra otro ambiente (lo lee el hook).
`mvn clean test`	Borra `target/` y vuelve a compilar desde cero. Úsalo si algo "raro" persiste.

Así se ve una ejecución correcta

```
Escenario: Consultar una reserva recién creada     # classpath:features/reservas.feature:11
  ✓ Dado que la API de reservas está disponible    # ReservaSteps.apiDisponible()
  ✓ Dado que existe una reserva a nombre de "Rosa" # ReservaSteps.existeReserva(String)
  ✓ Cuando consulto esa reserva                    # ReservaSteps.consultoReserva()
  ✓ Entonces el código de respuesta es 200         # ReservaSteps.codigoDeRespuesta(int)
  ✓ Y la respuesta contiene el campo "firstname"   # ReservaSteps.contieneCampo(String)
  … 3 escenarios más …
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time:  6.994 s
```

El reporte HTML

Al terminar, abre `target/cucumber-report.html` en el navegador. Verás cada escenario con sus pasos en verde o rojo, la duración y, si algo falló, el mensaje de error completo. Es el archivo que se adjunta como evidencia.

Desde VS Code

Abre la terminal integrada y usa los mismos comandos. Para trabajar en un solo escenario, ponle la etiqueta `@wip` y ejecuta `mvn test "-Dcucumber.filter.tags=@wip"`. La vista **Testing** ejecuta la suite completa con ▶.

"SKIPPED" NO ES UN ERROR

Cuando filtras por tags, Maven reporta como *Skipped* los escenarios excluidos. Con `@smoke` verás `Tests run: 4, Skipped: 3`: correcto.

PRUEBA DE HUMO DEL ENTORNO

Si llegaste a `BUILD SUCCESS`, tu equipo está listo: Java, Maven, las librerías y la conexión a internet funcionan. Lo siguiente ya es aprender a escribir buenos escenarios.

PRIMERA VEZ = MÁS LENTO

La primera ejecución descarga unos 40 MB de librerías. Las siguientes tardan entre 5 y 10 segundos.

CUANDO ALGO SALE EN ROJO

Errores típicos y cómo resolverlos

Los tres primeros aparecieron de verdad al preparar este manual. Léelos antes de pedir ayuda: el mensaje casi siempre dice exactamente qué falta.

```
java.lang.IllegalStateException: Cannot serialize object because no JSON serializer found in classpath.
Please put Jackson (Databind), Gson, Johnzon, or Yasson in the classpath.
```

CAUSA

Enviaste un `Map` u objeto como `body()` y no hay librería que lo convierta a JSON.

SOLUCIÓN

Agrega `jackson-databind` al `pom.xml` (ya está en el de este manual) y recarga Maven.

```
Expected status code <200> but was <418>.
```

CAUSA

`accept(ContentType.JSON)` envía la cabecera `Accept: application/json, application/javascript, text/javascript, text/json`. Restful Booker no la reconoce y responde **418 "I'm a teapot"**.

SOLUCIÓN

Envía un Accept exacto: `.accept("application/json")`. Lección: compara siempre lo que envía tu código con lo que envía `curl` o Postman.

```
[WARNING] The classpath resource selector 'features' should not be used to select features in a
package. Use the package selector with 'features' instead
```

CAUSA

Usaste `@SelectClasspathResource("features")`, la forma que aparece en muchos tutoriales antiguos.

SOLUCIÓN

Cámbialo por `@SelectPackages("features")`. Funciona igual y deja de advertir.

```
[ERROR] Fatal error compiling: error: invalid target release: 17
```

CAUSA

Maven corre con Java 8 porque `JAVA_HOME` apunta al JDK viejo.

SOLUCIÓN

Corrige `JAVA_HOME` (Paso 1), abre una terminal nueva y confirma con `mvn -v`.

```
The step 'creo una reserva para "Ana"…' is undefined. You can implement this step using the snippet(s)
below:
```

CAUSA

El texto del feature no coincide con ninguna expresión de los steps (una tilde, un espacio o una palabra distinta), o la clase está fuera del paquete `glue`.

SOLUCIÓN

Copia el snippet que propone Cucumber, o corrige el texto. Revisa que el paquete de los steps empiece por `com.canvia.qa`.

`[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0` · o bien · `No tests to run.`

CAUSA
El runner no se llama `…Test`, falta `junit-platform-suite` o el feature no está en `src/test/resources/features`.

SOLUCIÓN
Renombra la clase a `RunCucumberTest` y verifica la ruta exacta de la carpeta.

`CucumberException: class com.canvia.qa.steps.ReservaSteps does not have a public zero-argument constructor.`

CAUSA
La clase de steps recibe `Contexto` en el constructor pero no hay contenedor de inyección.

SOLUCIÓN
Agrega `cucumber-picocontainer` al `pom.xml`. No hace falta configurar nada más.

`Duplicate step definitions in ReservaSteps.java:… and OtroSteps.java:…`

CAUSA
Dos métodos tienen la misma expresión (copiar y pegar steps entre clases).

SOLUCIÓN
Deja una sola definición y reutilízala. Un step común ("el código de respuesta es…") va en una clase compartida.

`Could not transfer artifact io.rest-assured:rest-assured:pom:6.3.0 from/to central … Connect timed out / PKIX path building failed`

CAUSA
La red corporativa bloquea o inspecciona el tráfico hacia Maven Central.

SOLUCIÓN
Configura el proxy en `settings.xml` (Paso 2). Si el error es *PKIX*, pide a Mesa de Ayuda el certificado raíz e impórtalo con `keytool` en el JDK.

`[ERROR] Unknown lifecycle phase ".filter.tags=@smoke". You must specify a valid lifecycle phase…`

CAUSA
Solo en **PowerShell**: escribiste `-Dcucumber.filter.tags="@smoke"` y PowerShell parte el argumento en el primer punto.

SOLUCIÓN
Entrecomilla el argumento completo: `"-Dcucumber.filter.tags=@smoke"`. Esa forma funciona en los tres sistemas.

`Se ve "Característica" como "CaracterÃstica" o el step con tilde no se encuentra.`

CAUSA
Archivos guardados en otra codificación (ANSI / Windows-1252).

SOLUCIÓN
Guarda todo en UTF-8 (VS Code: la codificación aparece abajo a la derecha; haz clic y elige *Save with Encoding → UTF-8*) y conserva `sourceEncoding` en el pom.

`Expected status code <200> but was <404> al consultar /booking/1`

CAUSA
La API es pública y compartida: otra persona pudo borrar esa reserva. Los datos que no creaste tú cambian sin aviso.

SOLUCIÓN
Cada escenario crea sus propios datos (como el paso "que existe una reserva…"). Es una buena práctica, no solo un parche.

ANTES DE LA PRIMERA CLASE

Checklist: ¿está todo listo?

Marca cada casilla. Si todas están marcadas, llegas a la clase con el entorno funcionando y dedicamos el tiempo a aprender, no a instalar.

Entorno

- [ ] `java -version` responde 17 o 21.
- [ ] `JAVA_HOME` apunta al JDK nuevo (`echo $env:JAVA_HOME` en Windows, `echo $JAVA_HOME` en macOS/Linux).
- [ ] `mvn -v` muestra **Java version: 17** (o 21).
- [ ] VS Code abre el proyecto y muestra *"Java: Ready"*, sin errores en rojo.
- [ ] Las extensiones **Extension Pack for Java** y **Cucumber** están instaladas.
- [ ] `F12` sobre un paso del feature lleva al método Java.

Proyecto

- [ ] `mvn test` termina en **BUILD SUCCESS** con 4 pruebas.
- [ ] `mvn test "-Dcucumber.filter.tags=@smoke"` corre solo 1.
- [ ] Existe `target/cucumber-report.html` y se abre en el navegador.
- [ ] Cambiaste a propósito un `200` por `201` en el feature, viste el fallo en rojo y lo regresaste.
- [ ] Borraste una palabra de un paso, leíste el snippet de "undefined" y lo corregiste.

¿POR QUÉ ROMPER A PROPÓSITO?

Porque la primera vez que veas un fallo debe ser en un entorno controlado. Saber cómo se ve un error de aserción frente a uno de step indefinido te ahorra horas cuando ocurra de verdad.

Lo que viene en la presentación

CONCEPTOS
Qué es una API, verbos HTTP, códigos de estado y por qué probar APIs antes que pantallas.

BDD
Gherkin como idioma común entre negocio, desarrollo y QA; escenarios declarativos.

EJERCICIOS
Cinco retos progresivos sobre Restful Booker, terminando con una caza de bugs reales.

PROYECTO BASE

El proyecto completo de este manual se entrega como `api-bdd-restassured.zip` junto a este PDF. Descomprímelo, ábrelo en VS Code y ejecuta `mvn test` : si sale verde, tu instalación está validada.